

Université Polytechnique de Gitega
Faculté des Technologies de l'Information et de la Communication
Département de Génie Logiciel / BAC3
Année universitaire 2025-2026

PROJET LIBRE JAVA EE : REALISATION D'UNE APPLICATION WEB MULTI-TIERS PROFESSIONNELLE

1. Objectif pédagogique global

Vous allez concevoir et développer de A à Z une application web moderne en respectant strictement l'architecture multi-tiers Java EE telle qu'étudiée en cours. L'objectif est de maîtriser en profondeur les technologies du socle Java EE sans aucun framework additionnel ni raccourci interdit.

2. Pourquoi ce projet est important pour vous ?

C'est l'occasion de :

- comprendre vraiment comment fonctionnent les applications d'entreprise « sérieuses »
- pratiquer l'architecture en couches (présentation → métier → persistance)
- acquérir une rigueur professionnelle (séparation des responsabilités, injection, sécurité basique)

3. Technologies obligatoires et utilisation attendu

Technologie	Utilisation attendue	Remarques / interdictions
Servlet	Contrôleurs (doGet, doPost), forwarding, inclusion, gestion des requêtes/réponses	Obligatoire pour toute la logique de navigation
JSP	Vues dynamiques (affichage, formulaires)	Pas de scriptlets Java purs → utiliser JSTL + EL exclusivement
JSTL	Boucles (<c:forEach>), conditions (<c:if>, <c:choose>), formatage (<fmt:...>)	Obligatoire – zéro scriptlet autorisé dans les JSP
EJB	Logique métier (Session Beans Stateless ou Stateful)	Au moins 3–4 Session Beans avec méthodes métier
JPA	Entités, relations (@OneToMany, @ManyToOne...), EntityManager, JPQL simples	Au moins 6–8 entités bien normalisées
CDI	Injection dans Servlets, EJBs, et éventuellement helpers (@Inject)	Usage intensif : @Named non obligatoire mais scopes oui
Serveur d'applications	GlassFish 7+ (ou WildFly équivalent récent)	Tomcat seul interdit (pas de support EJB/CDI natif complet)

4. Technologies / approches formellement interdites

- JSF / Jakarta Faces / PrimeFaces / toute bibliothèque de composants faces
- API REST / JSON / JAX-RS
- Thymeleaf, Freemarker, ou tout autre moteur de template
- Spring (Boot ou non), Micronaut, Quarkus...

- JDBC direct (sauf si JPA impossible – mais non justifié ici)
- Frameworks frontend (React, Vue, htmx, Alpine...)
- Annotations de validation avancées (@Valid, groupes...)
- WebSocket, JMS, Batch, Concurrency, Mail...

5. Critères de validation du sujet (proposition à soumettre – 2–3 pages max)

1. Nom du projet + description courte (problème + public cible)
2. Liste des **rôles** (minimum 3 : ex. admin, utilisateur, gestionnaire)
3. **Fonctionnalités cœur** (6 à 10 max – priorisées)
 - inclure un **workflow avec plusieurs états** (ex. : brouillon → soumise → en attente → validée → refusée → archivée)
 - Au moins 2–3 formulaires multi-étapes ou avec pièces jointes simples (nom de fichier stocké en base)
4. Entités principales envisagées (minimum 6–8)

6. Exigences fonctionnelles minimales (MVP)

- Authentification basique (formulaire + session HttpSession + filtre Servlet)
- Gestion CRUD sur les entités principales via Servlets + JSP
- Logique métier dans des **Session Beans** (EJB) injectés via CDI
- Affichage de listes paginées ou filtrées avec JSTL
- Formulaires avec validation côté serveur (messages d'erreur simples)
- Relations entre entités (ex. Utilisateur → Demandes → Etat)
- Gestion d'un workflow d'états dans au moins un EJB (avec contrôle d'accès par rôle)

7. Livraisons attendues

- Projet WAR déployable sur serveur Glassfish
- Repository Git partagé (avec commits réguliers)
- Rapport technique (12–25 pages) :
 - Diagramme cas d'utilisation
 - Diagramme de classes (entités JPA + Session Beans)
 - Diagramme de séquence (2–3 scénarios clés)
 - Architecture (couches : web → EJB → JPA)
 - Extraits de code : Servlet + JSP/JSTL + EJB + entité JPA + injection CDI
 - Difficultés / choix techniques
- Démo fonctionnelle (live ou capture 10–15 min)
- Script SQL de création + jeu de données (10–20 enregistrements minimum)